

Final Task- Aircraft Model

1. Backend (Architecture, Equations, and Modules)

1.1. State and Control Vectors

The dynamic state vector X has 9 components, and the control vector U has 5 components, as defined below:

Variable	Description	Unit
$x1 = u$	Forward body velocity	m/s
$x2 = v$	Lateral body velocity	m/s
$x3 = w$	Vertical body velocity	m/s
$x4 = p$	Roll rate	rad/s
$x5 = q$	Pitch rate	rad/s
$x6 = r$	Yaw rate	rad/s
$x7 = \phi$	Roll angle	rad
$x8 = \theta$	Pitch angle	rad
$x9 = \psi$	Yaw angle	rad

The full simulation uses a 12-state vector that appends $[X_e, Y_e, Z_e]$ (NED position) to the 9 dynamic states. Altitude is recovered as $h = -Z_e$.

Variable	Description	Unit	Notes
$u1 = \delta A$	Aileron deflection	rad	± 0.2618 ($\pm 15^\circ$)
$u2 = \delta E$	Stabilizer deflection	rad	-0.436 to $+0.175$
$u3 = \delta R$	Rudder deflection	rad	± 0.2618 ($\pm 15^\circ$)
$u4 = dTH1$	Throttle engine 1	0–1	Right wing ($+7.94$ m)
$u5 = dTH2$	Throttle engine 2	0–1	Left wing (-7.94 m)

1.2. The xdot Function

The function `rcam_derivatives(X, U)` in `rcam_model.py` computes the full-time derivative of the state vector. It is structured in 10 explicit steps:

- **Step 1 (Airspeed and aerodynamic angles):** Total airspeed $V = \sqrt{u^2 + v^2 + w^2}$, angle of attack $\alpha = \text{atan2}(w, u)$, sideslip $\beta = \text{asin}(v/V)$. V is clamped to 0.1 m/s to prevent division by zero.
- **Step 2 (Aerodynamic coefficients):** Lift C_L includes wing-body contribution plus tail correction with downwash ($\varepsilon = 0.25 \cdot (\alpha - \alpha_0)$) and tail angle of attack $\alpha_t = \alpha - \varepsilon + \delta E + 1.3 \cdot q \cdot l_t/V$. Drag C_D is a parabola in $(C_L_{wb} - 0.45)$. Side force C_Y depends on β and rudder δR . Moment coefficients C_l, C_m, C_n include rate-damping terms $(l/V \cdot \omega)$ and control surface contributions.
- **Step 3 (Aerodynamic forces-body axes):** Dynamic pressure $\bar{q} = 0.5 \cdot \rho \cdot V^2$. Lift L , drag D , and side force Y are each coefficient $\times \bar{q} \times S$, then resolved into body axes F_{xA}, F_{yA}, F_{zA} using α and β .

- **Step 4 (Aerodynamic moments):** $LA = Cl \cdot \bar{q} \cdot S \cdot b$ (roll), $MA = Cm \cdot \bar{q} \cdot S \cdot \bar{c}$ (pitch), $NA = Cn \cdot \bar{q} \cdot S \cdot b$ (yaw).
- **Step 5 (Engine forces and moments):** Each engine produces thrust $Fi = dTHi \cdot m \cdot g$ along body $+x$. The moment arm cross-product $[XAPT_i, YAPT_i, ZAPT_i] \times [Fi, 0, 0]$ gives the engine moment. Symmetric throttles produce zero net yaw moments; asymmetric throttle creates a yawing moment of $7.94 \cdot F$ (arm length $\times$ force).
- **Step 6 (Gravity in body axes):** $Fx_{grav} = -m \cdot g \cdot \sin(\theta)$; $Fy_{grav} = m \cdot g \cdot \cos(\theta) \cdot \sin(\varphi)$; $Fz_{grav} = m \cdot g \cdot \cos(\theta) \cdot \cos(\varphi)$.
- **Step 7 (Total forces and moments):** $Fx, Fy, Fz = aerodynamic + engine + gravity$. $Mx, My, Mz = aerodynamic + engine$.
- **Step 8 (Translational equations of motion-Newton in body axes):** $\dot{u} = Fx/m + r \cdot v - q \cdot w$; $\dot{v} = Fy/m + p \cdot w - r \cdot u$; $\dot{w} = Fz/m + q \cdot u - p \cdot v$. The cross terms ($r \cdot v$, $p \cdot w$, etc.) are the Coriolis-like coupling from the rotating body frame.
- **Step 9 (Rotational equations of motion):** $\dot{\omega} = I^{-1} \cdot (M - \omega \times I \cdot \omega)$, solved with `numpy.linalg.solve`. The I_{xz} off-diagonal term couples roll and yaw; $\omega \times I \cdot \omega$ captures gyroscopic precession.
- **Step 10 (Euler-angle kinematics, matrix H):** $\dot{\varphi} = p + \sin(\varphi) \cdot \tan(\theta) \cdot q + \cos(\varphi) \cdot \tan(\theta) \cdot r$; $\dot{\theta} = \cos(\varphi) \cdot q - \sin(\varphi) \cdot r$; $\dot{\psi} = \sin(\varphi)/\cos(\theta) \cdot q + \cos(\varphi)/\cos(\theta) \cdot r$. A guard clamps $\cos(\theta)$ to 1×10^{-10} to prevent gimbal-lock blow-up at $\theta = \pm 90^\circ$.

1.3. Numerical Integration

The simulator uses a 4th-order Runge-Kutta (RK4) integrator implemented in `rk4_step(f, x, u, dt)` in `flight_main.py`. RK4 evaluates the derivative at four sub-points within each time step:

$$k1 = f(x, u); k2 = f(x + 0.5 \cdot dt \cdot k1, u); k3 = f(x + 0.5 \cdot dt \cdot k2, u); k4 = f(x + dt \cdot k3, u)$$

$$x_{new} = x + (dt/6) \cdot (k1 + 2 \cdot k2 + 2 \cdot k3 + k4)$$

The local truncation error is $O(dt^5)$, far superior to the forward-Euler $O(dt^2)$ used in Task 2. This is critical here because the roll damping eigenvalue of the RCAM is approximately 9.16 rad/s (roll time constant 0.11s). RK4 stability requires $|\lambda \cdot dt| < 2.79$, so dt must be less than 0.30s whenever aileron inputs are active. Scenarios 1 and 4 use $dt = 1\text{s}$ (no roll excitation); Scenarios 2 and 3 use $dt = 0.1\text{s}$.

1.4. Coordinate Frames and DCM

Three reference frames are used throughout the project:

- **Body frame:** x = forward (nose), y = *right wing*, z = *down*. All RCAM state variables are expressed here.
- **NED frame:** x = *North*, y = *East*, z = *Down*. Used for navigation and position output. Altitude $h = -Ze$.
- **GL frame (visualization only):** x = *East*, y = *North*, z = *Up*. Used internally by the 3D views so East appears consistently on the right side of the screen.

The body-to-NED Direction Cosine Matrix (DCM) is constructed in `build_dcm(euler_vector)` using the ZYX rotation sequence (yaw $\rightarrow$ pitch $\rightarrow$ roll):

$$C_{b_n} = Rz(\psi) \cdot Ry(\theta) \cdot Rx(\varphi)$$

Anybody-frame vector is transformed to NED as: $V_{NED} = C_{b_n} \cdot V_{body}$

The function *terminator()* wraps ϕ and ψ to $[-\pi, \pi]$, clamps θ to $[-\pi/2, \pi/2]$, and raises a gimbal-lock warning when $|\theta|$ approaches $\pm 90^\circ$.

2. Test Cases and Key Observations

Four scenarios were implemented and simulated. All share the same 9+3 state integration pipeline; only the initial conditions, control inputs, and time-step differ.

- **Scenario 1 (Untrimmed Baseline):** Initial state: $X_0 = [85, 0, 0, 0, 0, 0, 0, 0.1, 0]^T$ m/s and rad; control: $U = [0, -0.1, 0, 0.08, 0.08]^T$. The aircraft starts 0.1 rad nose-up with a negative elevator deflection and symmetric throttle. This combination is far from the trim point. The forward velocity u oscillates with a period of roughly 30–40 s — the classic phugoid mode, which is a long-period exchange between kinetic and potential energy. Pitch angle θ oscillates in phase, while altitude shows a slow descending trend because total thrust is insufficient to maintain level flight at 85 m/s with $dE = -0.1$ rad. All angular rates (p, q, r) remain near zero because no lateral or directional inputs are applied.
- **Scenario 2 — Aileron Pulse:** A $+5^\circ$ aileron deflection is applied for 2 seconds at $t = 30$ s via a $U_schedule(t)$ closure. This scenario requires $dt = 0.1$ s for RK4 numerical stability. The roll rate p spikes immediately at $t = 30$ s and recovers within ~ 2 –3 s after the pulse ends, consistent with the fast roll time constant (~ 0.11 s). The roll angle ϕ builds up during the pulse and decays slowly afterward due to roll-yaw coupling. The yaw rate r also responds through the I_{xz} inertia coupling and the aerodynamic $C_n \cdot dA$ contribution. The lateral velocity v grows gradually, indicating a developing sideslip. No significant phugoid disruption is observed in u or θ , confirming that the roll-yaw subsystem is dynamically decoupled from the longitudinal modes at this time scale.
- **Scenario 3 (Aileron Pulse + Engine Failure):** Built on Scenario 2 with an additional left engine failure ($dTH2 = 0$) at $t = 60$ s. With only the right engine (at $YAPT1 = +7.94$ m) producing thrust, a net yawing moment of $F_1 \times 7.94$ N·m acts on the aircraft nose-to-left. After $t = 60$ s the yaw rate r grows significantly and ψ deviates from its Scenario 2 baseline. The coupled roll-yaw dynamics enter a diverging spiral: ϕ and r reinforce each other through the dihedral and weathercock stability derivatives. The forward velocity u also decays because total available thrust is halved. This scenario clearly demonstrates the importance of differential thrust control and rudder coordination in asymmetric engine-out conditions.
- **Scenario 4 (PSO Trim at 78 m/s, NE heading):** The PSO algorithm in *ps_trim*($V=78, \psi=\pi/4$) finds the combination $[\alpha, \delta E, dTH]$ that drives all state derivatives to zero for straight-and-level trimmed flight. Once launched from the trim initial state, all nine state derivatives remain at ≈ 0 throughout the 180-second simulation. Forward velocity holds at 78 m/s, heading $\psi = 45^\circ$ (NE), altitude is constant, and all angular rates stay at zero. The position trace in NED shows a perfectly straight northeast trajectory. This confirms that the PSO converged to a true equilibrium point of the nonlinear RCAM model.

3. State Variable Plots

The GUI presents six tabs of time-series plots, one per state-variable group, updated in real time during animation playback. The figures below describe the expected behavior of each group for each scenario.

- **Body Velocities [u, v, w]:** In Scenario 1, u oscillates with the phugoid period (~35 s), w follows in quadrature, and v remains zero (no lateral excitation). In Scenario 2, v rises after the aileron pulse due to induced sideslip. In Scenario 3, u decreases after engine failure as total thrust drops. In Scenario 4, u is constant at 78 m/s and v, w ≈ 0 throughout.
- **Angular Rates [p, q, r]:** Scenario 1 shows q oscillating gently with the phugoid; p and r remain at zero. Scenario 2 shows a sharp p spike at t = 30 s decaying within ~2 s. Scenario 3 adds a growing r after t=60 s. Scenario 4 shows all rates identically zero.
- **Euler Angles [ϕ , θ , ψ]:** Scenario 1: θ oscillates with the phugoid; ϕ and ψ remain near zero. Scenario 2: ϕ builds up during the aileron pulse and relaxes. Scenario 3: ψ diverges after engine failure as the yaw moment is uncorrected. Scenario 4: all three angles are constant ($\phi = 0$, $\theta = \alpha_{trim} \approx 0.045$ rad, $\psi = \pi/4$).
- **Position [Xe, Ye, h]:** Scenario 1: altitude h decreases slowly due to insufficient thrust. Scenario 4: Xe and Ye grow linearly in the NE direction at 78 m/s, confirming straight-level flight. Scenarios 2–3 show lateral deviation developing after the aileron pulse and engine failure respectively.

4. Cost Function (PSO-Trim)

4.1. Trim Problem Formulation

Steady, straight-and-level trimmed flight requires that all nine state derivatives are simultaneously zero. The trim search reduces to finding three free variables (angle of attack α , elevator deflection δE , and throttle setting dTH) such that: $\dot{x} = rcam_derivatives(X_{trim}, U_{trim}) = 0$

The trim state is constructed as $X_{trim} = [V \cdot \cos(\alpha), 0, V \cdot \sin(\alpha), 0, 0, 0, 0, \alpha, \psi_{target}]^T$ and the trim control as $U_{trim} = [0, \delta E, 0, dTH, dTH]^T$ (ailerons and rudder at zero, both engines at equal throttle). This satisfies the geometrical constraints for wings-level flight automatically: $v = 0, p = q = r = 0, \phi = 0$.

4.2. Cost Function Definition

The cost function J is defined as the squared L2-norm of the full state derivative vector: $J(\alpha, \delta E, dTH) = ||\dot{x}||^2 = \sum_i (\dot{x}_i)^2$. This single scalar aggregates all nine equations of motion residuals. At perfect trim, J = 0. Any deviation from equilibrium in any channel (translational acceleration, angular acceleration, or Euler rate) is penalized quadratically. The squared norm is preferable to the absolute value because it is differentiable everywhere and penalizes large residuals more aggressively than small ones.

The constraints embedded in the search bounds are:

- $\alpha \in [-0.10, +0.35]$ rad, physical range of sustainable angles of attack for the RCAM.
- $\delta E \in [-0.436, +0.175]$ rad, actuator saturation limits of the horizontal stabilizer.

- $dTH \in [0.001, 1.0]$, throttle cannot be zero (engines remain on) or exceed full power.

4.3. PSO Algorithm Setup

Particle Swarm Optimization (PSO) was chosen because the cost function J is nonlinear and non-convex in the search space, gradient-based methods risk converging to local minima. PSO explores space stochastically through a swarm of candidate solutions that share information globally.

Parameter	Value / Description
Particles	80
Iterations	400
Inertia w	$0.9 \rightarrow 0.4$ (linear decay)
$c1$ (cognitive)	2.0
$c2$ (social)	2.0
α search range	$[-0.10, +0.35]$ rad
δE search range	$[-0.436, +0.175]$ rad
dTH search range	$[0.001, 1.0]$
Typical cost at convergence	1×10^{-26}
Trim result (typical)	$\alpha = 2.56^\circ, \delta E = -9.76^\circ, dTH = 0.077$

Each particle i holds a position $[\alpha_i, \delta E_i, dTH_i]$ and a velocity vector. At every iteration the velocity is updated as: $v_i \leftarrow w \cdot v_i + c1 \cdot r1 \cdot (pBest_i - x_i) + c2 \cdot r2 \cdot (gBest - x_i)$ where w is the inertia weight (decaying from 0.9 to 0.4 to balance global exploration early and local exploitation late), $pBest$ is the particle's personal best, $gBest$ is the global best, and $r1, r2$ are uniform random numbers in $[0, 1]$. The position is updated as $x_i \leftarrow x_i + v_i$ and clipped to the search bounds each iteration.

4.4. Convergence and Results

With 80 particles and 400 iterations the PSO typically converges to $J = 1 \times 10^{-26}$ within a few seconds, effectively reaching machine-precision zero. The resulting trim point for $V = 78 \text{ m/s}$ and $\psi = 45^\circ$ is:

- Angle of attack: $\alpha = 2.56^\circ$ (0.0447 rad).
- Elevator deflection: $\delta E = -9.76^\circ$ (-0.1704 rad), nose-up stabilizer input to maintain pitch trim.
- Throttle (each engine): $dTH = 0.077$, approximately 7.7% of maximum thrust.

These values confirm physically reasonable trim conditions: a small positive angle of attack generates the lift required at 78 m/s, the negative elevator counteracts the nose-down pitching moment from the wing lift, and low throttle is sufficient because the RCAM is a large, efficient civil aircraft model.

The code can be found in this GitHub repository:

https://github.com/SantiagoGirardoM17/Flight_Dynamics/tree/main/Final_Task